Calculadora de Edad: revisión integral en Python y C++

Alexis Maier

E.E.S.T. N.º 1 «Eduardo Ader», Vicente López

Laboratorio de Programación (LPR) — 5.º año 3.ª división, grupos A-B

Profesor: Mansilla Muñoz York Elías

21 de octubre de 2026

v1.0.0

Calculadora de Edad: revisión integral en Python y C++

Introducción

Esta actividad retoma los conceptos de lógica y algoritmos del año anterior y los aplica en dos lenguajes: Python, de alto nivel e interpretado, y C++, estructurado y compilado. El objetivo es escribir un programa que calcule la edad exacta de una persona a partir de su fecha de nacimiento, verificando antes que la fecha exista en el calendario real.

El informe describe la lógica del filtro de consistencia (años bisiestos y cantidad de días de cada mes), la corrección que se agregó a la plantilla (el condicional que resta un año si todavía no se cumplió años) y la evidencia de ejecución de ambas versiones. Todo el código está versionado en este repositorio respetando la estructura de carpetas pedida.

Desarrollo

Filtro de consistencia

La función `esBisiesto` aplica la regla (año % 4 == 0 y año % 100 != 0) o (año % 400 == 0). `esFechaValida` rechaza años fuera de 1900–2026, meses fuera de 1–12, días fuera de 1–31, días 31 en abril, junio, septiembre y noviembre, y en febrero admite 29 días solo si el año es bisiesto.

Completar la plantilla: ajuste de la edad

La plantilla calculaba `edad = anioA - anioN`, lo que sobreestima la edad de quien aún no cumplió años. Se completó con el condicional siguiente (idéntico en ambos lenguajes salvo sintaxis):

```
if (mesA < mesN || (mesA == mesN && diaA < diaN)) {
   edad--;
}
```

La fecha de referencia del ejercicio es 8/7/2026. Una persona nacida el 15/3/2008 ya cumplió años (edad 18); una nacida el 20/12/2008 todavía no (edad 17).

Comparación C++ y Python

La lógica es la misma, pero cambian la sintaxis y el modelo de ejecución: C++ exige declarar tipos, llaves y punto y coma y se compila a un ejecutable con `g++`; Python usa indentación, tipado dinámico y se ejecuta directamente con el intérprete. En Python,

además, `int(input())` lanza `ValueError` ante texto no numérico, por lo que se captura con `try/except`; en C++ el mismo caso deja el flujo `cin` en estado de error.

Evidencias

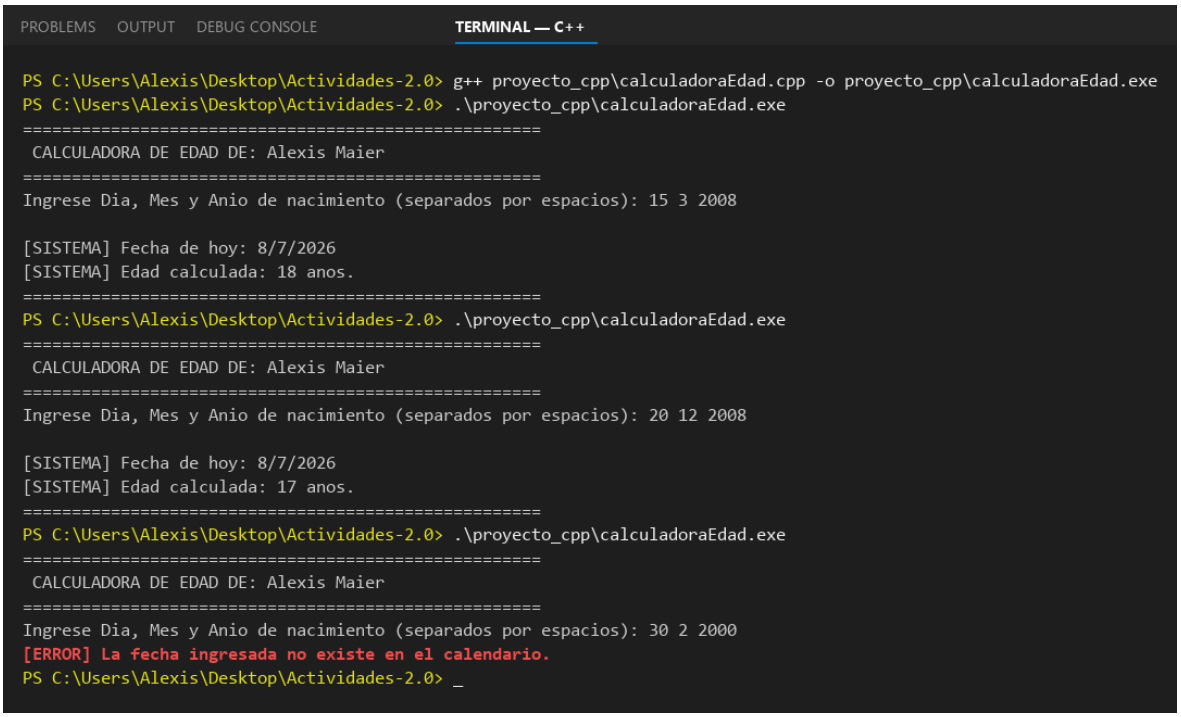


Figura 1. Compilación y ejecución de la versión en C++ (fechas válidas y una inexistente).



Figura 2. Ejecución de la versión en Python (fecha válida, cumpleaños el día de hoy y 29/2/2023 inexistente).

Conclusión

Ambas versiones producen los mismos resultados, lo que confirma que la lógica del problema es independiente del lenguaje. El filtro de consistencia evita datos imposibles y el ajuste por cumpleaños corrige un error de un año que sería fácil de pasar por alto. La experiencia sirvió para comparar la verbosidad de C++ con la concisión de Python y para ordenar el proyecto en un repositorio con carpetas separadas por lenguaje y documentación.

Referencias

cplusplus.com. (s. f.). *C++ reference*. https://cplusplus.com/reference/

Python Software Foundation. (s. f.). *The Python tutorial*. https://docs.python.org/3/tutorial/

Mansilla Muñoz, Y. E. (2026). *Actividad 1: De revisión integral (Python & C++)* [Guía de actividad]. E.E.S.T. N.º 1 «Eduardo Ader».